

Table 2. Characteristics of benchmark tasks.

Task	Input	Logic Program	Kind	#Rules	Provenance
Pathfinder	Image	Check if two dots are connected by a sequence of dashes.	Diff.	2	diff-top-1-proofs
PacMan-Maze	Image	Plan optimal next step by finding safe path from actor to goal.	Diff.	14	diff-top-1-proofs
HWF	Images	Parse and evaluate formula over recognized symbols.	Diff.	13	diff-top-1-proofs
CLUTTR	Text	Deduce kinship by recursively applying composition rules.	Diff.	3	diff-top-1-proofs
Prob. Static Analysis	Code	Compute alarms with severity via probabilistic static analysis.	Prob.	28	minmaxprob
RNA SSP	RNA	Parse an RNA sequence according to a context-free grammar.	Prob.	28	prob-top-1-proofs
Transitive Closure	Graph	Compute transitive closure of a directed graph.	Disc.	2	unit
Same Generation	Graph	Compute graph vertices that are in the “same generation”.	Disc.	2	unit
CSPA	Graph	A context sensitive pointer analysis.	Disc.	10	unit

separately will fall short due to work imbalances. Further, the symbolic program requires support for floating-point data and floating-point arithmetic operations.

CLUTRR CLUTRR is a natural language reasoning task about family kinship relations [39]. The input contains a natural language passage about a family with each sentence in the passage hinting at kinship relations. The goal is to infer the relationship between a given pair of characters; however, the target relation is not stated explicitly in the passage and it must be deduced through a reasoning chain. The most difficult problem in the evaluation dataset requires reasoning through a chain of length 10.

Probabilistic Static Analysis This benchmark extends static program analysis with probabilistic inputs. Specifically, analysis inputs are annotated with probabilities to reflect the system’s confidence. These probabilities are propagated to the output and used to rank results in order to decrease the visibility of false positives [42].

RNA SSP This task performs RNA Secondary Structure Prediction (SSP) using the ArchiveII [40] dataset. RNA SSP discovery is of widespread interest in the medical community, as the secondary structure of RNA molecules is crucial for understanding their function. Our neurosymbolic solution uses a Datalog program to parse an RNA sequence according to a context-free grammar, given probabilistic input from a transformer model. The dataset consists of a set of 475 RNA sequences of length 28 to 175.

Transitive Closure This benchmark computes the reachability of nodes in a graph using discrete reasoning. We use graphs from the SNAP graph repository [23] that take at least 1 second to process.

Same Generation This benchmark computes which nodes in a directed graph are the same distance from a common ancestor, i.e. which nodes are in the “same generation”. We use the same graphs as in the Transitive Closure benchmark.

Context Sensitive Pointer Analysis (CSPA) This benchmark computes a context sensitive pointer analysis for a program. We mirror the evaluation of GDLog [44] both in the Datalog program that we use and the input graphs.

6.2 Baselines

We compare Lobster to several other systems, as shown previously in Figure 2. While no prior system matches Lobster’s feature set, comparisons to more limited systems help us gauge Lobster’s performance across a range of use-cases.

Scallop supports differentiable reasoning with provenance semirings like Lobster, but supports only batch-level CPU multicore parallelism and therefore struggles to scale with problem and data complexity. We do not evaluate on Deep-ProbLog [29], a similar CPU-only system supporting differentiable and probabilistic reasoning, as previous work has shown that Scallop’s performance is uniformly superior [27].

ProbLog [13] provides discrete and probabilistic reasoning, but does not support GPU acceleration or CPU multi-threading. Notably, ProbLog executes logic programs via stable model semantics, indicating it may find certain programs easier or harder than other baselines, which are all based on bottom-up search. Additionally, while ProbLog has been used to study approximate probabilistic reasoning [35], approximate inference is not implemented in the publicly released ProbLog. Therefore, in our experiments ProbLog performs exact inference, as opposed to the more scalable approximate inference that Lobster and Scallop perform.

FVLog [43] supports only discrete reasoning, but does leverage GPU acceleration. FVLog is specialized for a different sort of workload than Lobster: it targets large batch analysis jobs that may span minutes, whereas Lobster emphasizes running the same program multiple times as a component of a neurosymbolic model. Notably, FVLog does not offer a Datalog front-end and query planner, meaning that all FVLog programs are human-written, low-level, relational algebra programs. As FVLog is faster than prior GPU discrete Datalog systems (like GDLog [44]), we compare only to FVLog for brevity.

Soufflé [36] is a state-of-the-art, multicore, CPU Datalog engine. While it does not support differentiable or probabilistic reasoning, comparing with Soufflé helps reveal the benefits of GPU acceleration versus CPU optimizations.

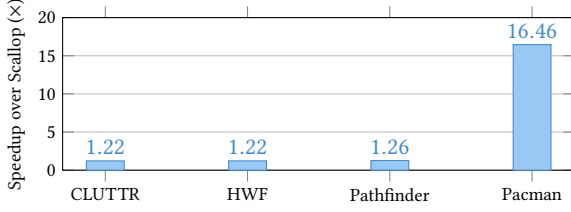


Figure 8. Lobster’s speedup over Scallop on training tasks.

6.3 Lobster for Training

To evaluate the extent to which Lobster improves performance in the training pipeline, we compare the total training time of Lobster against Scallop, the only other system that supports neurosymbolic training. For each task, training is run until convergence rather than a pre-determined number of epochs, and therefore takes a task-specific number of epochs. However, for a given task, both Lobster and Scallop take the same number of epochs, as they are executing the same Datalog program and produce identical results.

The results in Figure 8 reveal that Lobster can achieve significant speedups in end-to-end training time compared to Scallop, ranging from 1.2x to 16x. Figure 8 includes the cost of neural computations, which are already heavily optimized on GPU hardware via Pytorch [33] and unaffected by Lobster, so Amdahl’s Law limits the potential end-to-end speedup. Pacman is an exception as it performs extensive symbolic computation which Lobster can greatly accelerate.

6.4 Lobster for Inference

Beyond training of neurosymbolic models, we also evaluate Lobster’s performance on a range of neurosymbolic inference tasks. In neurosymbolic inference, the neural component is pre-trained. For example, with Pacman a pre-trained neural classifier identifies in-game objects, but the symbolic program still needs to be run to determine the path to the goal for each game board. We report average times across a set of samples: CLUTRR processes relationship graphs from 13 text passages, HWF evaluates 160 formulas of length 13, Pathfinder is evaluated on a set of 1216 images, and PacMan-Maze solves 50 mazes on a 15x15 grid.

Figure 9 shows that Lobster can obtain significant speedups over Scallop. Compared to results from training (Figure 8), during inference benchmarks like CLUTTR and Pathfinder spend significantly less time in neural computation which leads to larger speedups with Lobster. Notably, the speedups for Pacman is less for inference than for training. We believe this is because the trained model solves each maze in fewer steps, making the symbolic computation a smaller fraction of the total runtime.

Next we explore how well Lobster scales to larger problem sizes. We choose the benchmarks that can be scaled most naturally: for Pathfinder we increase the resolution of the analysis and for Pacman the maze size. We choose

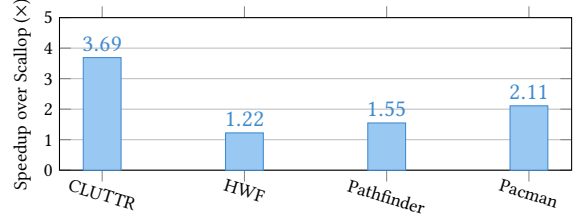
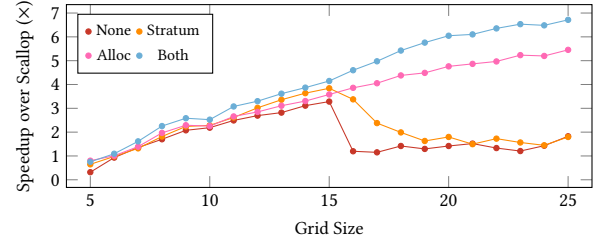
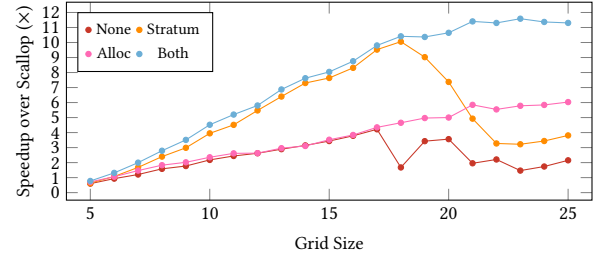


Figure 9. Lobster’s speedup over Scallop on neurosymbolic inference tasks.



(a) Lobster’s scalability on Pacman.



(b) Lobster’s scalability on Pathfinder.

Figure 10. Lobster’s scalability on Pathfinder and Pacman in the presence of various optimizations. “None” indicates no optimizations, “Stratum” includes the stratum scheduling heuristic of Section 5.3, “Alloc” includes the allocation optimizations of Section 4.1, and “Both” includes both optimizations.

Scallop as the baseline as it is the only system that supports these neurosymbolic workloads. Further, we only consider the symbolic computation time to more precisely measure Lobster’s improvement. In Figure 10 we see that Lobster handles large problem sizes better than Scallop, although the speedup plateaus as the problem size becomes large enough to saturate GPU memory bandwidth. Figure 10 also shows an ablation study that justifies some of the optimizations in Lobster—without allocation optimization and stratum scheduling, performance rapidly degrades to near equal with Scallop on problem sizes greater than 20.

Beyond neurosymbolic workloads, we also run two **probabilistic** inference workloads: Probabilistic Static Analysis (PSA) and RNA Secondary Structure Prediction (RNA SSP). As these two workloads require only probabilistic (not differentiable) reasoning, we attempted to run them with ProbLog

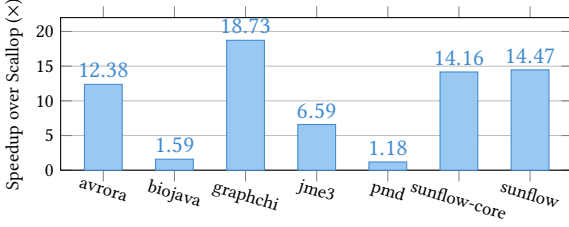


Figure 11. Lobster’s speedup over Scallop on Probabilistic Static Analysis.

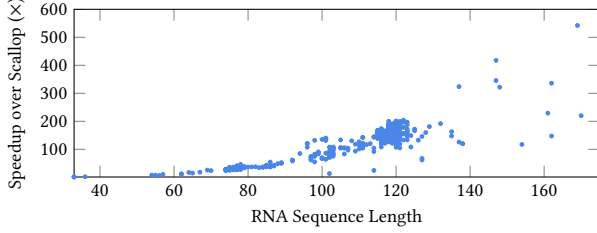


Figure 12. Lobster speedup over Scallop on RNA SSP.

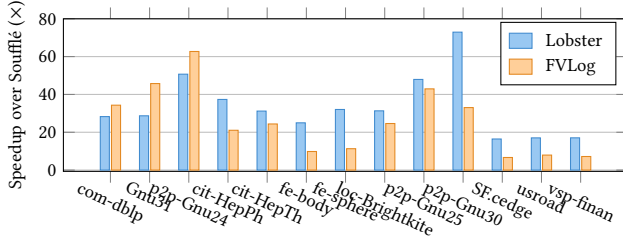


Figure 13. Speedup over Soufflé on Transitive Closure.

[13]. However, all of the PSA and RNA SSP runs hit our 2-hour timeout, except for PSA on sunflow-core which was 60% slower than Scallop and 30x slower than Lobster. As stated in Section 6.2, we believe this is explained by ProbLog’s exact probabilistic inference.

Figure 11 shows that, on the PSA benchmark, Lobster again offers significant speedups over Scallop when performing static analysis across a range of source programs. With RNA SSP (Figure 12), on the very shortest sequence (28 base pairs) Lobster is 40% slower than Scallop. However, on all of the other sequences Lobster achieves a speedup, frequently by two orders of magnitude. The speedup correlates with sequence length, making Lobster even more valuable on longer sequences which are, generally, of greater biological interest.

Finally, we examine Lobster’s performance on **discrete reasoning** tasks that require neither differentiable nor probabilistic reasoning. We first run the Transitive Closure benchmark from FVLog [43] on a range of input graphs [23]. For these inputs, ProbLog always hit our 2-hour timeout, and Scallop (when not timing out) has a 30-90x slowdown over Soufflé, so we omit results for those systems. Figure 13 shows

Table 3. The runtime of Lobster versus FVLog on the Same Generation task. “OOM” indicates that the system ran out of memory.

Dataset	Lobster (s)	FVLog (s)
fe-sphere	3.91	12.99
CA-HepTH	2.16	6.40
ego-Facebook	0.53	OOM
Gnu31	OOM	OOM
fe_body	10.17	21.17
loc-Brightkite	1.45	OOM
SF.cedge	14.01	23.72
com-dblp	OOM	OOM
usroad	OOM	OOM
fc_ocean	2.17	4.67
vsp_finan	OOM	90.10

Table 4. The runtime of Lobster versus FVLog on the CSPA task.

Dataset	Lobster (s)	FVLog (s)
httpd	3.61	2.57
linux	1.81	3.91
postgres	3.32	4.39

results for Lobster and FVLog and demonstrates that, despite Lobster’s generality, it offers competitive performance even against more specialized systems, consistently beating the CPU-only Soufflé and often surpassing the GPU-accelerated FVLog. We attribute this to Lobster’s adoption of APM. FVLog in particular lacks any IR and thus forgoes the opportunities afforded by IR-level optimizations (Section 4).

Since FVLog is the sole competitive system, we only run the remaining two discrete tasks, Same Generation and CSPA, on Lobster and FVLog, displaying the results in Table 3 and Table 4 respectively. We observe that for the Same Generation task, Lobster is at least twice as fast on each dataset and that there are multiple datasets that Lobster processes which FVLog runs out of memory on. The one exception is the vsp_finan dataset, where Lobster runs out of memory while FVLog finishes in 90 seconds. We believe this is due to the fact that Lobster is more general and therefore requires more memory to store intermediate results. For the CSPA task, Lobster and FVLog are approximately matched, with Lobster exhibiting a geometric mean speed up of 1.27x over FVLog.

7 Related Work

While there is a wealth of work on GPU-acceleration for SQL in both research and industry (e.g., [19, 34]), we focus